interop & declaration files	import JS modules, explain `any`.	declaration files, leverage `unknown` and `never` for safer interop.	complex JS libraries; automate API type generation from OpenAPI/GraphQL schemas.
Tooling & build	Set up `tsc`, configure a basic `tsconfig.json`.	Integrate with bundlers (Webpack, Vite), enable source maps and incremental builds.	Architect monorepos, automate type checking in CI/CD pipelines, enforce code quality gates.

Armed with an understanding of type safety and tools, we are ready to tackle interview questions where these concepts come into play.

Question: "Why are TypeScript's declaration files (`.d.ts` files) necessary?"

Analysis: This question tests a candidate’s understanding of TypeScript’s static type system and its integration with JavaScript code. It evaluates whether the candidate can explain the purpose of declaration files, their role in enabling type safety and tooling support, and how they facilitate the use of both TypeScript and JavaScript libraries within the same project.

Answer: TypeScript declaration files (`.d.ts` files) contain only type declarations—such as interfaces, types, functions, and classes—without implementation code. They describe the shape and structure of JavaScript code so that TypeScript can understand its types, even when the actual implementation is not written in TypeScript.

Declaration files are essential because they bridge untyped JavaScript code and TypeScript’s static type system, enabling static type checking, IntelliSense, autocompletion, and refactoring support, which brings type safety to JavaScript and allows seamless integration of external npm modules into TypeScript projects. Many modern npm packages bundle their declaration files automatically, making it easy for users to get type safety and IDE support without handling TypeScript source directly. Shipping